实验 2　流程控制实验

2.1 实验目的

（1）掌握复合语句、if 语句、switch 语句的使用，熟练掌握 for、while、do-while 三种基本的循环控制语句的使用，掌握重复循环技术，了解转移语句与标号语句。

（2）练习循环结构 for、while、do-while 语句的使用。

（3）练习转移语句和标号语句的使用。

（4）使用集成开发环境中的调试功能：单步执行、设置断点、观察变量值。

2.2 实验内容及要求

1．程序改错

下面的实验 2-1 程序是合数判断器（合数指自然数中除了能被 1 和本身整除外，还能被其它数整除的数），在该源程序中存在若干语法和逻辑错误。要求对该程序进行调试修改，使之能够正确完成指定任务。

```
/*  实验2-1改错题程序：合数判断器*/
#include <stdio.h>
int main( )
{
    int i, x, k, flag = 0;
    printf("本程序判断合数，请输入大于1的整数，以Ctrl+Z结束\n");
while (scanf("%d", &x) !=EOF) {
    for(i=2,k=x>>1;i<=k;i++)
        if (!x%i) {
            flag = 1;
            break;
        }
    if(flag=1) printf("%d是合数", x);
    else printf("%d不是合数", x);
    }
        return 0;
}
```

解答：

错误修改：

1) 第 5 行只有 flag 初始化，i，x，k 未被初始化，应改为

int i = 0, x = 0, k = 0, flag = 0;

2) 第 9 行 if (!x%i) 语法错误：应改为

! (x % i)

3) 第 13 行 if(flag=1) 是赋值语句，而不是判断语句，应改为

if (flag == 1);

4) flag 在循环中没有每次初始化，上一次的 flag 的值会对下一次循环产生影响，应在第 7 行和第 8 行之间添加：

flag = 0

2．程序修改替换

（1）修改实验 2-1 程序，将内层两出口的 for 循环结构改用单出口结构，即不允许使用 break、goto 等非结构化语句。

源代码清单：

```
/* 实验 2-1 改错题程序：合数判断器*/
#include <stdio.h>
int main( ){
    int i=0, x=0, k=0, flag = 0;
    printf("本程序判断合数，请输入大于 1 的整数，以 Ctrl+Z 结束\n");

    while (scanf("%d", &x) !=EOF) {
        flag = 0;
        for(i=2,k=x>>1; i<=k && !flag; i++) {
                if (x % i == 0) {
                    flag = 1;
                }
        }
        if(flag==1) printf("%d 是合数", x);
        else printf("%d 不是合数", x);
    }
        return 0;
}
```

（2）修改实验 2-1 程序，将 for 循环改用 do-while 循环。

源代码清单：

```
#include <stdio.h>
int main( ){
        int i=0, x=0, k=0, flag = 0;
        printf("本程序判断合数，请输入大于 1 的整数，以 Ctrl+Z 结束\n");
        while (scanf("%d", &x) !=EOF) {
                flag = 0;
                i = 2; k = x >> 1;
                if (i <= k) {
                        do {
                                if (!(x % i)) {
                                        flag = 1;
                                        break;
                                }
                                i++;
                        } while (i <= k);
                }
                if(flag==1) printf("%d 是合数", x);
                else printf("%d 不是合数", x);
        }
        return 0;
}
```

（3）修改实验 2-1 程序，将其改为纯粹合数求解器，求出所有的 3 位纯粹合数。一个合数去掉最低位，剩下的数仍是合数；再去掉剩下的数的最低位，余留下来的数还是合数，这样反复，一直到最后剩下一位数仍是合数，这样的数称为纯粹合数。

源代码清单：

```
/*  实验 2-1 改错题程序：纯粹合数判断器*/
    #include <stdio.h>
    int removeOnes(int n) {
        int ones = n%10;
        int removedn = (n-ones)/10;
        return removedn;
```

```
}

int isComposite(int n) {
    int i, k;
    for(i=2, k=n>>1; i<=k; i++) {
        if (n % i == 0) {
            return 1;
        }
    }
    return 0;
}

int main( ){
    for (int i=100; i<=999; i++) {
        if (isComposite(i) &&
            isComposite(removeOnes(i)) &&
            isComposite(removeOnes(removeOnes(i)))) {
            printf("%d\n", i);
        }
    }
}
```

3．程序设计

（1） 假设工资税金按以下方法计算：x ＜ 1000 元，不收取税金；1000 ≤ x ＜ 2000，收取 5%的税金；2000 ≤ x ＜ 3000，收取 10%的税金；3000 ≤ x ＜ 4000，收取 15%的税金；4000 ≤ x ＜ 5000，收取 20%的税金；5000 ≤ x，收取 25%的税金。（注意税金的计算按照阶梯计税法，比如，工资为 4500，那么税金=1000*5% + 1000*10% + 1000*15% + 501*20%）。编写一个程序,输入工资金额，输出应收取税金额度，要求分别用 if 语句和 switch 语句来实现。

解答思路：算法流程如图 2-1 所示：

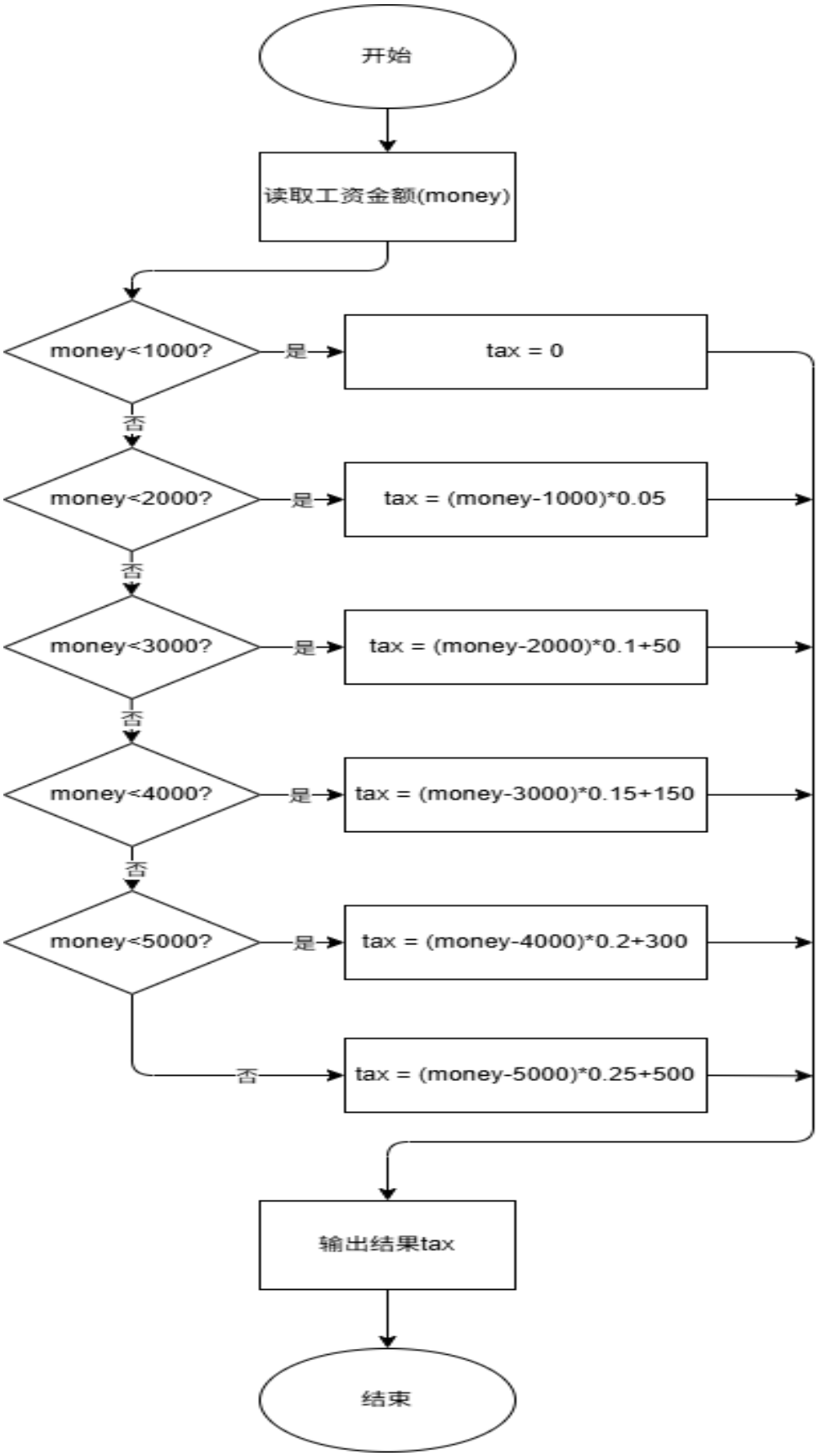

图 2-1 编程题 1 的程序流程图

2）源程序清单：

```
/* if */
#include <stdio.h>
int main(){
    int money, tax;
    printf("请输入工资金额：");
    scanf("%d", &money);

    if (money < 1000){
        tax = 0;
    }else if (money <2000){
        tax = (money - 1000) * 0.05;
    }else if (money <3000){
        tax = (money - 2000) * 0.1 + 50;
    }else if (money < 4000){
        tax = (money - 3000) * 0.15 + 150;
    }else if (money < 5000){
        tax = (money - 4000) * 0.2 + 300;
    }else{
        tax = (money - 5000) * 0.25 + 500;
    }
    printf("工资税金：%d\n", tax);
    return 0;
}

/* switch */
#include <stdio.h>
int main(){
    int money, tax;
    printf("请输入工资金额：");
    scanf("%d", &money);

    switch (money / 1000){
        case 0:
            tax = 0;
            break;
        case 1:
            tax = (money - 1000) * 0.05;
            break;
```

```
        case 2:
            tax = (money - 2000) * 0.1 + 50;
            break;
        case 3:
            tax = (money - 3000) * 0.15 + 150;
            break;
        case 4:
            tax = (money - 4000) * 0.2 + 300;
            break;
        default:
            tax = (money - 5000) * 0.25 + 500;
            break;
    }

    printf("工资税金：%d\n", tax);
    return 0;
}
```

3）测试

（a）测试数据:

测试用例	程序输入	理论结果	实际结果
用例 1	500	0	0
用例 2	1500	25	25
用例 3	2500	100	100
用例 4	3500	225	225
用例 5	4500	400	400
用例 6	5500	625	625

（b）对应测试用例的运行结果如图 2-2 所示。

图 2-2 编程题 1 的测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

(2) 输入一段以!结尾的短文(最多 5 行,每行不超过 50 个字符)，要求将它复制到输出，复制过程中将每行一个以上的空格字符用一个空格代替。

1） 解题思路：算法流程如图 2-3 所示。

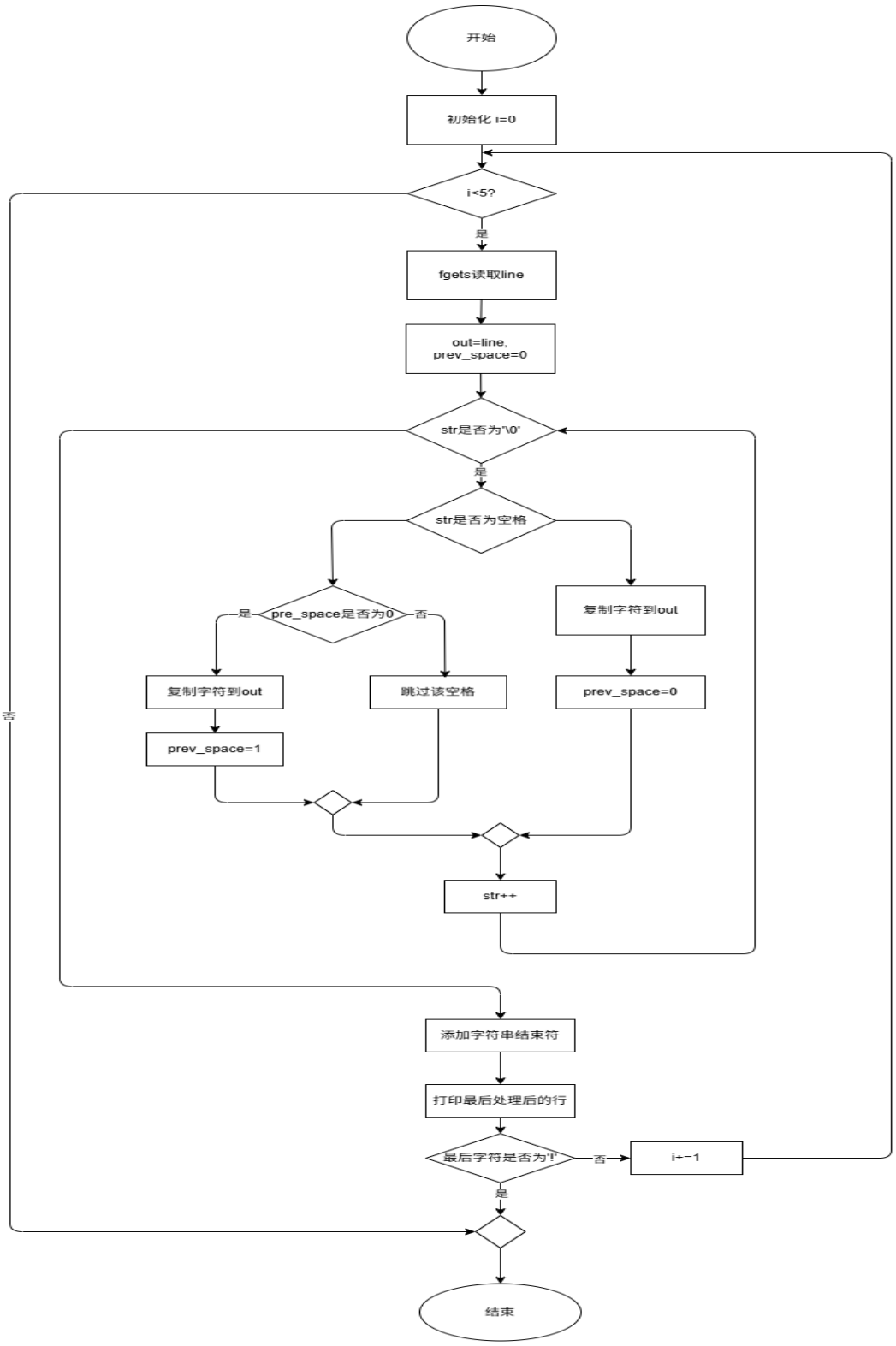

图 2-3 编程题 2 的程序流程图

2）程序清单

```
#include <stdio.h>
#include <string.h>
void clean_spaces(char *str) {
    char *out = str;
    int prev_space = 0;
    while (*str) {
        if (*str == ' ') {
            if (!prev_space) {
                *out++ = ' ';
                prev_space = 1;
            }
        } else {
            *out++ = *str;
            prev_space = 0;
        }
        str++;
    }
    *out = '\0';
}
int main() {
    char line[50];
    for (int i = 0; i < 5; i++) {
        fgets(line, sizeof(line), stdin);
        clean_spaces(line);
        printf("%s", line);
        if (line[strlen(line) - 1] == '!' ) {
            break;
        }
    }
}
```

3）测试

（a） 测试数据：

如表 2-2 所示。

表 2-2 编程题 2 的测试数据

测试用例	程序输入	理论结果	运行结果

用例 1	Mary had a little lamb . It is a good pet for Mary The lamb did not live in the water The lamb did not fly away The lamb liked to look at books!	Mary had a little lamb . It is a good pet for Mary The lamb did not live in the water The lamb did not fly away The lamb liked to look at books!	Mary had a little lamb . It is a good pet for Mary The lamb did not live in the water The lamb did not fly away The lamb liked to look at books!

（b） 对应测试测试用例的运行结果如图 2-4 所示。

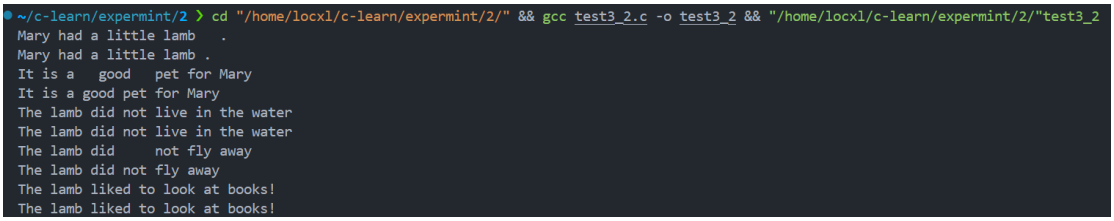

图 2-4 编程题 2 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（3）打印如下的杨辉三角形。

```
                    1                    /*第 0 行 */
                  1   1                  /*第 1 行 */
                1   2   1                /*第 2 行 */
              1   3   3   1
            1   4   6   4   1
          1   5   10  10  5   1
        1   6   15  20  15  6   1
      1   7   21  35  35  21  7   1
    1   8   28  56  70  56  28  8   1
  1   9   36  84  126 126 84  36  9   1
```

第 i 行第 j 列位置的数据值可以由组合 C_i^j 表示，而 C_i^j 的计算如下：

$$C_i^0 = 1 \qquad (i=0,1,2,\ldots)$$

$$C_i^j = C_i^{j-1} * (i - j + 1) / j \qquad (j=0,1,2,3,\ldots,i)$$

根据以上公式，采用顺推法编程，输入最后一行的编号 N（0<=N<=6），要求输出金字塔效果的杨辉三角形。

特别要注意空格的数目，一位数之间是 3 个空格，两位数之间有 2 个空格，3 位数之间只有一个空格。第 N 行行首是 N 个空格。每行末尾是 3 个空格和换

行。

解答：

1） 解题思路：算法流程如图 2-5 所示。

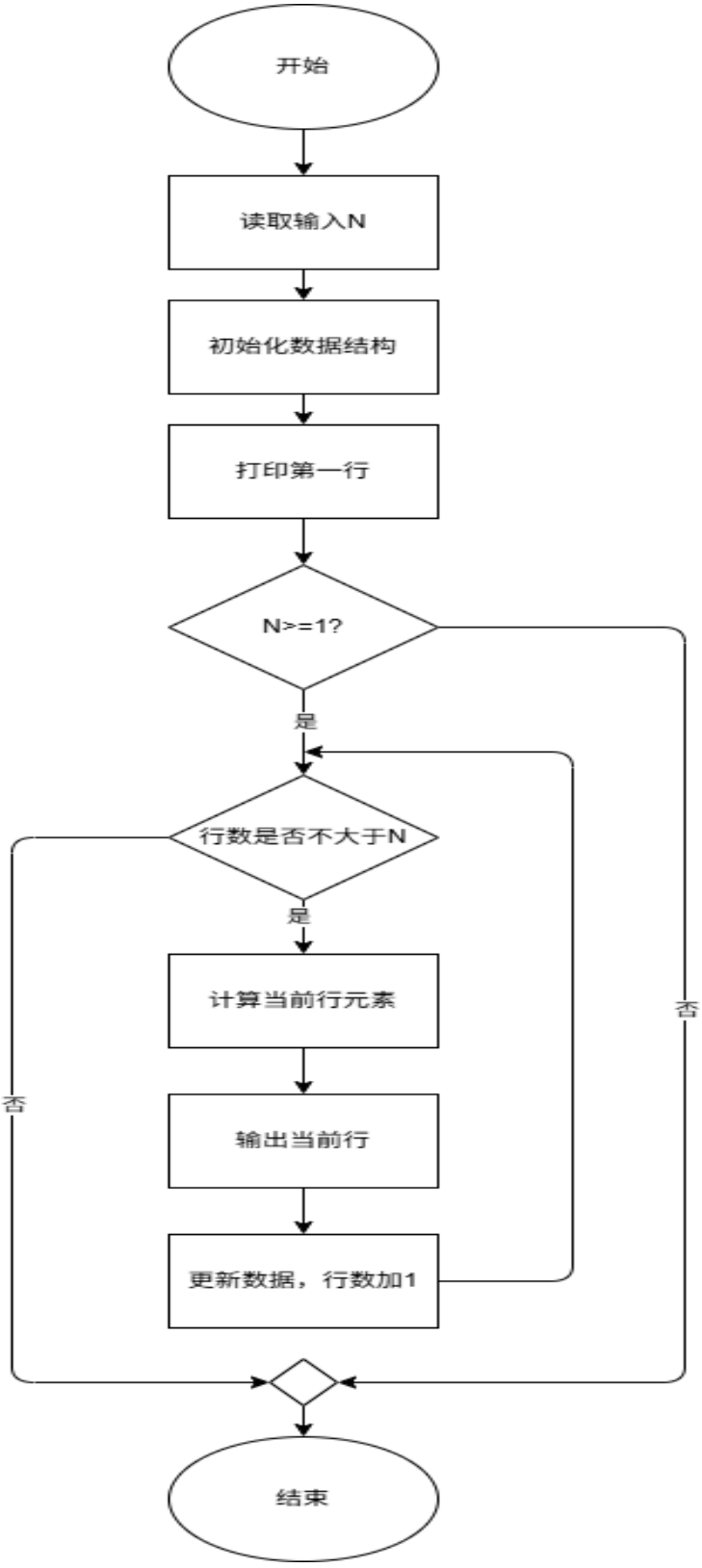

图 2-5 编程题 3 的程序流程图

2）程序清单：

```
#include <stdio.h>
int main() {
    int N;
    scanf("%d", &N);
    int prev_row[7] = {0};
    prev_row[0] = 1;
    int spaces = 3 * N;
    for (int s = 0; s < spaces; s++) {
        printf(" ");
    }
    printf("1       \n");
    if (N >= 1) {
        int curr_row[7] = {0};
        for (int i = 1; i <= N; i++) {
            curr_row[0] = 1;
            for (int j = 1; j < i; j++) {
                curr_row[j] = prev_row[j-1] + prev_row[j];
            }
            curr_row[i] = 1;

            spaces = 3 * N - 2 * i;
            for (int s = 0; s < spaces; s++) {
                printf(" ");
            }

            for (int j = 0; j <= i; j++) {
                printf("%d", curr_row[j]);
                if (j < i) {
                    if (curr_row[j] < 10) {
                        printf("       ");
                    } else {
                        printf("      ");
                    }
                } else {
```

```
                printf("        \n");
            }
        }

        for (int k = 0; k <= i; k++) {
            prev_row[k] = curr_row[k];
        }
    }
}
return 0;
}
```

3）测试

（a） 测试数据：

如表 2-3 所示。

表 2-3 编程题 3 的测试数据

测试用例	程序输入	理论结果	运行结果
用例 1	1	1 1 1	1 1 1
用例 2	3	1 1 1 1 2 1 1 3 3 1	1 1 1 1 2 1 1 3 3 1
用例 3	6	1 1 1 1 2 1 1 3 3 1 1 4 6 4 1 1 5 10 10 5 1 1 6 15 20 15 6 1	1 1 1 1 2 1 1 3 3 1 1 4 6 4 1 1 5 10 10 5 1 1 6 15 20 15 6 1

（b） 对应测试测试用例的运行结果如图 2-6 所示。

```
~/c-learn/expermint/2 ❯ cd "/home/locxl/c-learn/expermint/2/" && gcc test3_3.c -o test3_3 && "/home/locxl/c-learn/expermint/2/"test3_3
1
   1
 1   1
~/c-learn/expermint/2 ❯ cd "/home/locxl/c-learn/expermint/2/" && gcc test3_3.c -o test3_3 && "/home/locxl/c-learn/expermint/2/"test3_3
3
        1
      1   1
    1   2   1
  1   3   3   1
~/c-learn/expermint/2 ❯ cd "/home/locxl/c-learn/expermint/2/" && gcc test3_3.c -o test3_3 && "/home/locxl/c-learn/expermint/2/"test3_3
6
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5   10  10  5   1
    1   6   15  20  15  6   1
```

图 2-6 编程题 3 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（4）625 这个数很特别，625 的平方等于 390625，其末 3 位也是 625。请编程输出所有这样的 3 位数：它的平方的末 3 位是这个数本身。要求这些数字从小到大排列，每个数字单独占一行。

1）解题思路：算法流程如图 2-7 所示。

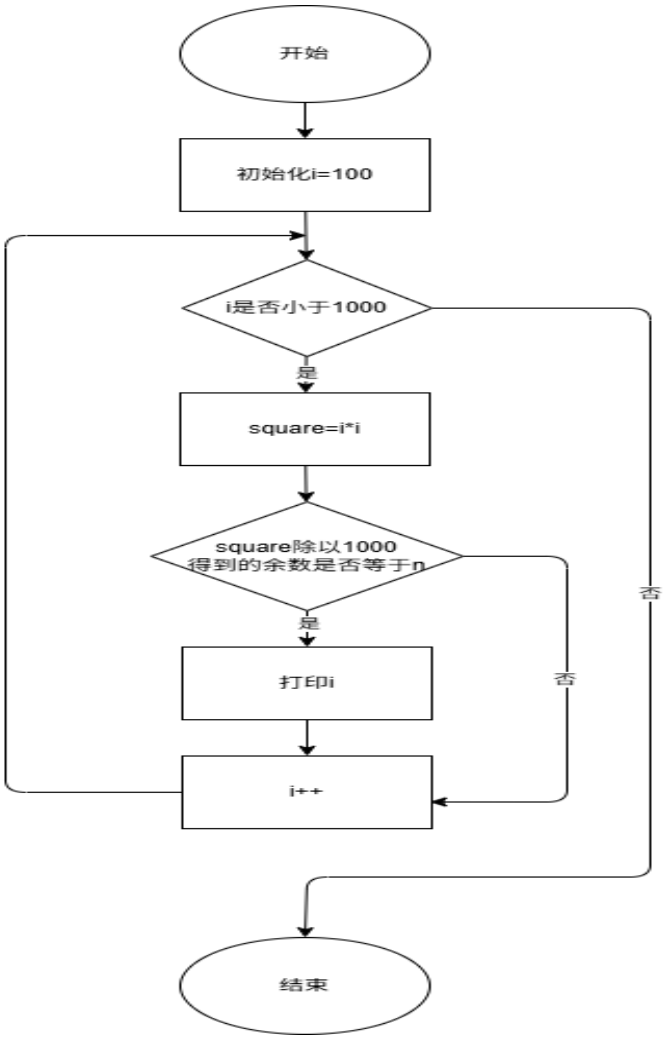

图 2-7 编程题 4 的程序流程图

2） 程序清单：

```
#include <stdio.h>

int is_special(int n){
    int square = n * n;
    if (square % 1000 == n ){
        return 1;
    }
    return 0;
}
int main() {
    for (int i = 100; i < 1000; i++) {
        if (is_special(i)) {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

3） 测试

（a） 测试数据：

如表 2-4 所示

测试用例	理论结果	运行结果
用例 1	376 625	376 625

（b） 对应测试测试用例的运行结果如图 2-8 所示.

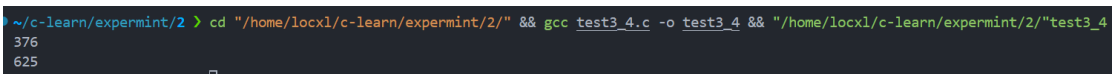

图 2-8 编程题 4 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

2.3 实验小结

在实验过程中，主要遇到程序逻辑错误与语法问题。例如，在合数判断器实验中，首次调试发现 if(flag=1)误用赋值操作符，导致判断逻辑失效；同时 flag 未在循环中重置，导致后续输入结果异常。通过逐行检查代码、利用调试工具观察变量值，最终修正了条件判断与变量初始化问题。此外，将多出口循环改为单出口结构时，需调整循环条件以避免冗余判断，提高了代码可读性。

实验中通过分析问题根源，深刻体会到流程控制语句的严谨性。例如，工资税金计算时，若未合理设计分支逻辑易导致计算偏差，而通过阶梯式条件判断与调试验证，成功实现功能；在处理字符串空格时，需关注字符逐个处理与状态标记（如 prev_space）的正确性。这些实践强化了对循环、条件语句逻辑结构的理解。

本次实验提升了代码调试与逻辑分析能力，认识到初始化变量、避免隐式错误（如赋值替代比较）的重要性。同时，通过多方案实现（如 if 与 switch 对比）体会到不同控制结构的适用场景，为后续复杂程序设计打下基础。